

Esercizi del Capitolo 2

Classi, oggetti, campi e metodi

LEGGI PRIMA DI COMINCIARE

Qui si scrivono classi per davvero. Ti serve tutto il capitolo 2: `class`, il costruttore `__init__`, `self`, i campi, i metodi, `__str__` e `__repr__`, e la differenza fra dati del singolo oggetto e dati della classe.

Dove lavorare

Se usi un **notebook**, crea una cella nuova per ogni esercizio e premi `Shift + Invio`. Se usi **PyCharm**, crea un file nuovo per ogni esercizio (per esempio `esercizio_1.py`) e premi il pulsante verde *Run*.

Come sono fatte le tracce

Ogni esercizio ti dice esattamente quali campi e quali metodi deve avere la classe, con i nomi precisi da usare. Poi ti dà un **blocco di prova** da incollare in fondo, che non va modificato, e l'**output esatto** che il tuo programma deve stampare.

Usa i nomi indicati senza cambiarli: il blocco di prova li chiama con quei nomi, e se ne cambi uno il programma non parte. L'output atteso è il tuo strumento di autocorrezione: se combacia riga per riga, l'esercizio è corretto.

Difficoltà	Che cosa aspettarti	Tempo indicativo
● ● ● Guidato	Una classe semplice, con i metodi già descritti uno per uno.	20 – 30 minuti
● ● ● Autonomo	Compaiono i dati di classe e un po' di logica dentro i metodi.	40 – 50 minuti
● ● ● Impegnativo	Classe completa più il programma che la usa, partendo dalla tua progettazione.	50 – 70 minuti

TRE ERRORI CHE FARAI QUASI CERTAMENTE

- Dimenticare `self` fra le parentesi quando definisci un metodo. Il messaggio sarà `takes 0 positional arguments but 1 was given`.
- Dimenticare `self.` davanti a un campo o a un altro metodo, dentro la classe. Il messaggio sarà `name '...' is not defined`.
- Scrivere una lista direttamente nel corpo della classe invece che dentro `__init__`: la lista finirebbe condivisa da tutti gli oggetti.

La classe Libro

La situazione

Una piccola biblioteca vuole tenere traccia dei suoi libri e sapere quali sono in prestito. Devi scrivere la classe che rappresenta un singolo libro.

Che cosa devi fare

Scrivi una classe di nome `Libro` con i campi e i metodi elencati qui sotto. Usa esattamente questi nomi.

I **campi**, tutti creati dentro `__init__`:

Campo	Da dove arriva	Significato
<code>titolo</code>	parametro del costruttore	Il titolo del libro.
<code>autore</code>	parametro del costruttore	Il nome dell'autore.
<code>anno</code>	parametro del costruttore	L'anno di pubblicazione, come numero intero.
<code>disponibile</code>	non è un parametro	Vale <code>True</code> appena il libro viene creato: un libro nuovo è sempre disponibile.

I metodi:

Metodo	Che cosa deve fare
<code>presta()</code>	Se il libro è disponibile, lo segna come non disponibile e restituisce <code>True</code> . Se era già in prestito non cambia nulla e restituisce <code>False</code> .
<code>restituisce()</code>	Il contrario: se il libro era in prestito lo rimette disponibile e restituisce <code>True</code> ; se era già disponibile non cambia nulla e restituisce <code>False</code> .
<code>eta(anno_corrente)</code>	Riceve un anno e restituisce quanti anni ha il libro, cioè la differenza fra l'anno ricevuto e l'anno di pubblicazione.
<code>__str__()</code>	Restituisce una riga fatta così: <code>Titolo (Autore, anno) - stato</code> , dove <code>stato</code> è la parola <code>disponibile</code> oppure <code>in prestito</code> a seconda della situazione. Guarda l'output atteso per il formato preciso.

Blocco di prova

Incolla questo blocco dopo la classe, senza modificarlo.

```
a = Libro("Il nome della rosa", "Umberto Eco", 1980)
b = Libro("La coscienza di Zeno", "Italo Svevo", 1923)

print(a)
print(b)
print("eta del primo libro nel 2026:", a.eta(2026), "anni")

print("presto il primo libro:", a.presta())
print(a)
print("provo a prestarlo di nuovo:", a.presta())

print("lo restituisco:", a.restituisce())
print(a)
print("il secondo libro e' ancora disponibile?", b.disponibile)
```

Output atteso

```
Il nome della rosa (Umberto Eco, 1980) - disponibile
La coscienza di Zeno (Italo Svevo, 1923) - disponibile
eta del primo libro nel 2026: 46 anni
presto il primo libro: True
Il nome della rosa (Umberto Eco, 1980) - in prestito
provo a prestarlo di nuovo: False
lo restituisco: True
Il nome della rosa (Umberto Eco, 1980) - disponibile
il secondo libro e' ancora disponibile? True
```

SUGGERIMENTI

- Il costruttore riceve tre parametri oltre a `self`: `def __init__(self, titolo, autore, anno):`. Il campo `disponibile` invece non si riceve, si imposta e basta: `self.disponibile = True`.
- In `presta()` controlla prima la situazione e poi cambiala, così: se `self.disponibile` è `False`, esci subito con `return False`; altrimenti metti `self.disponibile = False` e restituisci `True`.
- Dentro `__str__` ricordati che devi **restituire** la stringa con `return`, non stamparla con `print`.
- Per costruire la riga di `__str__` puoi unire i pezzi con il `+`, ma allora l'anno va convertito in testo con `str(self.anno)`, perché non si possono sommare numeri e stringhe.
- Se ti serve un ripasso, guarda la prova 16 del capitolo 2 per `__str__` e la prova 6 per il costruttore.

HAI FINITO QUANDO

- ☐ il tuo output combacia riga per riga con quello atteso;
- ☐ prestare due volte lo stesso libro restituisce `False` la seconda volta;
- ☐ il secondo libro non è stato toccato dalle operazioni fatte sul primo;
- ☐ tutti e quattro i campi vengono creati dentro `__init__`.

La carta fedeltà

La situazione

Un supermercato ti chiede il programma delle carte fedeltà. Ogni cliente ha la sua carta con i suoi punti, ma le **regole del programma fedeltà sono uguali per tutti**: quanti punti si guadagnano per ogni euro e quanti punti servono per il premio. Anche il conteggio delle carte emesse è unico per tutto il supermercato.

Questa distinzione — dati del singolo oggetto contro dati condivisi da tutti — è il cuore dell'esercizio.

Che cosa devi fare

Scrivi una classe di nome `CartaFedelta`.

I dati della classe, scritti nel corpo della classe e non dentro `__init__`, perché valgono per tutte le carte:

Nome	Valore	Significato
<code>PUNTI_PER_EURO</code>	<code>2</code>	Punti guadagnati per ogni euro speso.
<code>SOGLIA_PREMIO</code>	<code>100</code>	Punti necessari per ritirare un premio.
<code>carte_emesse</code>	<code>0</code>	Contatore di quante carte sono state create in tutto.

I campi del singolo oggetto, creati dentro `__init__`:

Campo	Significato
<code>intestatario</code>	Il nome del cliente, ricevuto come parametro.
<code>punti</code>	I punti accumulati. Una carta nuova parte da <code>0</code> .
<code>codice</code>	Il codice della carta. Si costruisce da solo: la lettera <code>F</code> , un trattino, e il numero della carta appena emessa. La prima carta creata avrà codice <code>F-1</code> , la seconda <code>F-2</code> , e così via.

I metodi:

Metodo	Che cosa deve fare
<code>acquista(euro)</code>	Il cliente spende una certa cifra. Calcola i punti guadagnati moltiplicando gli euro (arrotondati per difetto a numero intero con <code>int()</code>) per <code>PUNTI_PER_EURO</code> , li aggiunge ai punti della carta e restituisce quanti punti ha guadagnato in questo acquisto.
<code>punti_mancanti()</code>	Restituisce quanti punti mancano per arrivare alla soglia del premio. Se il cliente ha già raggiunto o superato la soglia, deve restituire <code>0</code> e non un numero negativo.
<code>usa_premio()</code>	Se il cliente ha almeno <code>SOGLIA_PREMIO</code> punti, gliene toglie esattamente <code>SOGLIA_PREMIO</code> e restituisce <code>True</code> . Altrimenti non cambia nulla e restituisce <code>False</code> .
<code>__str__()</code>	Restituisce una riga fatta così: <code>codice intestatario: N punti</code> . Guarda l'output atteso per il formato preciso.
<code>__repr__()</code>	Restituisce <code>CartaFedelta('nome del cliente')</code> , virgolette semplici comprese. È la forma che compare quando stampi una <i>lista</i> di carte.

Blocco di prova

```

anna = CartaFedelta("Anna Rossi")
luca = CartaFedelta("Luca Bianchi")

print(anna)
print(luca)
print("carte emesse:", CartaFedelta.carte_emesse)

print("Anna spende 30 euro e guadagna", anna.acquista(30), "punti")
print("le mancano", anna.punti_mancanti(), "punti al premio")
print("puo' usare il premio?", anna.usa_premio())

print("Anna spende altri 25 euro e guadagna", anna.acquista(25), "punti")
print(anna)
print("le mancano", anna.punti_mancanti(), "punti al premio")
print("puo' usare il premio?", anna.usa_premio())
print(anna)

print("la carta di Luca e' rimasta a", luca.punti, "punti")
print([anna, luca])

```

Output atteso

```

F-1 Anna Rossi: 0 punti
F-2 Luca Bianchi: 0 punti
carte emesse: 2
Anna spende 30 euro e guadagna 60 punti
le mancano 40 punti al premio

```

```
puo' usare il premio? False
Anna spende altri 25 euro e guadagna 50 punti
F-1 Anna Rossi: 110 punti
le mancano 0 punti al premio
puo' usare il premio? True
F-1 Anna Rossi: 10 punti
la carta di Luca e' rimasta a 0 punti
[CartaFedelta('Anna Rossi'), CartaFedelta('Luca Bianchi')]
```

SUGGERIMENTI

- Per aumentare il contatore condiviso devi scrivere `CartaFedelta.carte_emesse = CartaFedelta.carte_emesse + 1`, cioè nominando la **classe**. Se scrivessi `self.carte_emesse = ...` creeresti un contatore personale di quella singola carta, e il totale resterebbe fermo.
- Costruisci il codice *dopo* aver aumentato il contatore, così la prima carta prende il numero 1.
- I valori condivisi puoi leggerli con `self.PUNTI_PER_EURO`: dall'oggetto si vedono benissimo, ed è la scrittura più comoda dentro i metodi.
- In `punti_mancanti()` calcola la differenza e poi controlla: se è minore di zero restituisci `0`.
- In `usa_premio()` fai prima il controllo e poi la sottrazione, come in `presta()` dell'esercizio 1.
- Se ti serve un ripasso sui dati condivisi, guarda le prove 17 e 18 del capitolo 2.

HAI FINITO QUANDO

- ☐ il tuo output combacia riga per riga con quello atteso;
- ☐ le due carte hanno codici diversi (`F-1` e `F-2`) e `carte_emesse` vale 2;
- ☐ i punti di Anna non hanno mai toccato la carta di Luca;
- ☐ l'ultima riga mostra la forma `CartaFedelta('...')` e non degli indirizzi di memoria: se vedi `<__main__...>` ti manca `__repr__`.

Il magazzino, questa volta con le classi

La situazione

Questo esercizio chiude il lavoro cominciato negli esercizi del capitolo 1. Lì avevi scritto il magazzino con i dizionari e le funzioni (esercizio 2), e poi avevi **progettato su carta** la classe `Prodotto` (esercizio 3). Adesso quella classe la scrivi davvero.

Prima di iniziare, riprendi il tuo schema di progettazione e tienilo accanto: servirà a confrontare quello che avevi previsto con quello che l'esercizio ti chiede.

SE NON HAI FATTO GLI ESERCIZI DEL CAPITOLO 1

Puoi comunque svolgere questo esercizio: qui sotto trovi tutto quello che serve. Ti perderai però il confronto fra la tua progettazione e la versione richiesta, che è la parte più interessante.

Che cosa devi fare

Scrivi una classe di nome `Prodotto`.

I dati della classe (nel corpo della classe, condivisi da tutti i prodotti):

Nome	Valore	Significato
<code>IVA</code>	<code>0.22</code>	L'aliquota IVA, uguale per tutti i prodotti.
<code>prodotti_creati</code>	<code>0</code>	Contatore di quanti prodotti sono stati creati in tutto.

I campi del singolo prodotto (dentro `__init__`): `nome`, `prezzo` e `quantita`. Attenzione a un dettaglio: `quantita` deve essere un parametro **facoltativo** che vale `0` se chi crea il prodotto non lo indica.

I metodi:

Metodo	Che cosa deve fare
<code>valore_totale()</code>	Restituisce il prezzo moltiplicato per la quantità.
<code>prezzo_con_iva()</code>	Restituisce il prezzo aumentato dell'IVA, arrotondato a due cifre decimali con <code>round(..., 2)</code> .
<code>disponibile()</code>	Restituisce <code>True</code> se c'è almeno un pezzo, <code>False</code> altrimenti.
<code>vendi(quantità)</code>	Se ci sono abbastanza pezzi diminuisce la quantità e restituisce <code>True</code> ; altrimenti non cambia nulla e restituisce <code>False</code> . Questa volta non deve stampare niente .
<code>riifornisci(quantità)</code>	Aumenta la quantità del numero indicato. Non restituisce niente.
<code>__str__()</code>	Restituisce <code>Nome - prezzo EUR x quantità</code> . Guarda l'output atteso per il formato preciso.
<code>__repr__()</code>	Restituisce <code>Prodotto('nome')</code> , virgolette semplici comprese.

Blocco di prova

```

mouse = Prodotto("Mouse", 25.0, 4)
tastiera = Prodotto("Tastiera", 45.0, 2)
monitor = Prodotto("Monitor", 180.0)

magazzino = [mouse, tastiera, monitor]

for p in magazzino:
    print(p)

print("prodotti creati:", Prodotto.prodotti_creati)
print("prezzo del Mouse con IVA:", mouse.prezzo_con_iva())
print("Monitor disponibile?", monitor.disponibile())

print("vendo 2 Mouse:", mouse.vendi(2))
print("vendo 5 Tastiere:", tastiera.vendi(5))

monitor.rifornisci(3)
print("dopo il rifornimento:", monitor)

totale = 0
for p in magazzino:
    totale = totale + p.valore_totale()
print("valore del magazzino:", totale)

print("solo i disponibili:", [p for p in magazzino if p.disponibile()])

```

Output atteso

```
Mouse - 25.0 EUR x 4
Tastiera - 45.0 EUR x 2
Monitor - 180.0 EUR x 0
prodotti creati: 3
prezzo del Mouse con IVA: 30.5
Monitor disponibile? False
vendo 2 Mouse: True
vendo 5 Tastiere: False
dopo il rifornimento: Monitor - 180.0 EUR x 3
valore del magazzino: 680.0
solo i disponibili: [Prodotto('Mouse'), Prodotto('Tastiera'), Prodotto('Monitor')]
```

Parte finale — Il confronto

Quando il programma funziona, rispondi per iscritto a queste tre domande. Valgono quanto il codice.

1. Confronta il blocco di prova con quello dell'esercizio 2 del capitolo 1. Indica **due differenze** nel modo in cui si scrivono le operazioni, e di' quale delle due versioni si legge meglio.
2. Nel capitolo 1 avevi una funzione `valore_magazzino(lista)`. Qui non c'è: il totale viene calcolato dal ciclo `for` nel blocco di prova. Questo conferma o smentisce la risposta che avevi dato alla parte D dell'esercizio 3 del capitolo 1?
3. Il blocco di prova crea il Monitor scrivendo `Prodotto("Monitor", 180.0)`, senza indicare la quantità. Perché funziona? E che cosa succederebbe se anche `prezzo` fosse facoltativo?

SUGGERIMENTI

- Per rendere facoltativa la quantità, il costruttore si scrive `def __init__(self, nome, prezzo, quantita=0):`. I parametri con valore predefinito vanno sempre **dopo** quelli obbligatori.
- Per l'IVA: il prezzo aumentato del 22% si calcola come `self.prezzo * (1 + self.IVA)`.
- Il contatore `prodotti_creati` si aumenta dentro `__init__`, nominando la classe: `Prodotto.prodotti_creati = Prodotto.prodotti_creati + 1`.
- L'ultima riga del blocco di prova usa una *lista* di prodotti: per vederla stampata bene ti serve `__repr__`, non basta `__str__`.
- Attenzione all'ultima riga dell'output: dopo il rifornimento il Monitor ha 3 pezzi, quindi risulta disponibile e compare nell'elenco.

HAI FINITO QUANDO

- ☐ il tuo output combacia riga per riga con quello atteso;
- ☐ creare un prodotto senza indicare la quantità funziona e la mette a zero;
- ☐ la vendita fallita della Tastiera non ne ha cambiato la quantità;
- ☐ l'ultima riga mostra `Prodotto('...')` e non degli indirizzi di memoria;
- ☐ hai risposto per iscritto alle tre domande della parte finale.